CS4475 Fall 2026 Lab Report
Elian Pena Ramos Submitted on 10/17/2026
epenaramos@miners.utep.edu

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

A pa2 branch was created from the pa1-syscalls branch, so it already had the time command, wait2() and CPUS := 1 from PA1, and it was pushed to the repository. ps.c was added in the user directory and _ps was added to UPROGS in the Makefile. The getprocs() system call returns the number of processes that are currently in use and fills the array of struct pstat that ps passes to it, with one entry for each process.

The files kernel/pstat.h, kernel/proc.h, user/user.h, user/usys.pl, kernel/syscall.h, kernel/syscall.c, kernel/defs.h, kernel/sysproc.c, kernel/proc.c were modified.

-kernel/pstat.h: struct pstat was added with the fields pid, state, size, ppid and name. The enum procstate was moved here from proc.h, because ps.c does not include proc.h and without the enum the state field had an incomplete type. The file has a guard (#ifndef PSTAT_H) so it is not defined twice when a file includes it more than once.

-kernel/proc.h: the line of enum procstate was replaced with #include "pstat.h" so the kernel and the user programs use the same definition of the states.

-user/user.h: the lines struct pstat; and int getprocs(struct pstat*); were added. The first one is a forward declaration so the prototype can use struct pstat*, and the prototype lets user programs call getprocs().

-user/usys.pl: entry("getprocs") was added to generate the assembly stub in usys.S

-kernel/syscall.h: #define SYS_getprocs 25 was added to give the new system call a number that is unique

-kernel/syscall.c: the lines extern uint64 sys_getprocs(void); and [SYS_getprocs] sys_getprocs were written in the syscalls[] table to add the system call information of getprocs().

-kernel/defs.h: int kgetprocs(uint64); was added in the proc.c section to make kgetprocs() visible to sysproc.c

-kernel/sysproc.c: sys_getprocs() was added. It uses argaddr(0,) to get the address of the array that ps passed and then calls kgetprocs() with it.

-kernel/proc.c: kgetprocs() was implemented. It goes through the proc[] array and skips the UNUSED slots. For every other process it takes wait_lock and p->lock, and copies the pid, state, sz, the pid of the parent (0 for init, since it has no parent) and the name into a local struct pstat. After releasing the locks, either_copyout() copies the struct into the user's array, since the kernel can not write directly into a user address. The function returns the number of processes that were copied.

All changes were pushed.

```
xv6 kernel is booting

pinit: starting sh
$ ps
exec pps failed
$ ps
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
4       running         0       no aging        16384   2       ps
$
```

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.
The modified ps.c is in the user directory in the pa2 branch. For this task a priority field was added to struct proc and the getpriority() and setpriority() system calls were implemented. Higher numbers mean higher priority, processes can have priorities from 0 to 49.
The files kernel/param.h, kernel/proc.h, kernel/proc.c, kernel/pstat.h, kernel/syscall.h, kernel/syscall.c, kernel/sysproc.c, user/usys.pl, user/user.h, user/ps.c were modified.
-kernel/param.h: #define MAXPRIORITY 49 and #define MAXEFFPRIORITY 99 were added so the limits are not written as numbers in different places.
-kernel/proc.h: int priority; was added in struct proc, in the section of fields that are protected by p->lock.
-kernel/proc.c: the line p->priority = 0; was added in allocproc() after p->state = USED; This makes new processes begin at 0 instead of the value of a previous process that used the same slot, and since userinit() calls allocproc(), init starts with priority 0. In fork() the line np->priority = p->priority; was added after np->sz = p->sz; so a child inherits the priority of its parent.
-kernel/syscall.h, kernel/syscall.c, user/usys.pl, user/user.h: SYS_getpriority 25 and SYS_setpriority 26 were added with their extern lines, their entries in the syscalls[] table, entry("getpriority") and entry("setpriority"), and the prototypes int getpriority(void); and int setpriority(int);
-kernel/sysproc.c: sys_getpriority() returns the priority of the process that made the call (myproc()). sys_setpriority() uses argint(0,) to get the new priority and returns -1 if it is not between 0 and MAXPRIORITY, otherwise it sets the priority and returns 0. Both read or write the field while holding p->lock.
The changes that had to be made to the getprocs() system call were adding int priority; to struct pstat in kernel/pstat.h and adding the line ps.priority = p->priority; in kgetprocs() with the other fields that are copied. The rest of the system call stayed the same since it already copies the whole struct.
-user/ps.c: a PRIO column was added in the header and %d with uproc[i].priority was added in the same position of the printf that prints each process.
A test program, prioritytest.c, was created in the user directory and added to UPROGS. It prints its starting priority, tries setpriority(60) which should fail, sets its priority to 20, and then calls fork(). The child prints its priority to show it inherited 20, changes it to 30, and then turns itself into ps with exec(), so ps shows the parent with 20 and itself with 30. After wait(0), the parent prints its priority again to show that the change of the child did not affect it.

All changes were committed and pushed.
Results

```
xv6 kernel is booting

init: starting sh
$ prioritytest
priority at start: 0
setpriority(60) failed
priority now: 20
child priority 20: 20
child priority after change: 30
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
3       sleeping        20      no aging        16384   2       prioritytest
4       running         30      no aging        16384   3       ps
parent priority is still: 20
$
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

-kernel/proc.h: uint readytime; was added in struct proc next to priority. It stores the tick when the process last became RUNNABLE.

-kernel/proc.c: the line readytime = ticks; was added right after every place where a process state is changed to RUNNABLE. These places are userinit() for init, fork() for a new child, yield() when the timer takes the CPU away from a running process, wakeup() when a sleeping process gets what it was waiting for, and kkill() when a sleeping process is killed. They were found with grep -n "= RUNNABLE" kernel/*.c so none was missed. ticks is read without tickslock, because the clock interrupt holds tickslock when it calls wakeup() and not doing so causes problems with xv6. p->readytime = 0; was also added in allocproc() to clear the value of a previous process.

-kernel/pstat.h and kgetprocs(): uint readytime; was added to struct pstat and ps.readytime = p->readytime; was added in kgetprocs() so ps can get it.

-user/ps.c: an AGE column was added. The age is calculated in ps and not in the kernel. Before the loop, the current time is stored once with int current = uptime(); which uses the same ticks counter that readytime comes from. Then, for each process that is RUNNABLE, the age is current - readytime, which is how many ticks the process has been waiting in the ready queue. For any other state the age has no meaning, since the process is not waiting for the CPU, so a no aging is printed instead.

To test it, executions.c was created and added to UPROGS. It is a loop that only counts, so it always wants the CPU. Three spin processes were started in the background and then ps was run, so the executions processes show runnable with their age. Since the round-robin scheduler was still being used, the ages were small because every process gets the CPU again after a few ticks.

All changes were committed and pushed.

Results:

```
init: starting sh
$ executions &
$ executions &
$ executions &
$ ps
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
9       running         0       no aging        16384   2       ps
$ $ executions &; executions &; executions &; ps
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
10      running         0       no aging        16384   2       ps
12      runnable        0       0       86016   1       sh
14      runnable        0       0       86016   1       sh
16      runnable        0       0       86016   1       sh
$ exec $ failed
```

Task 4. (15 pts) Implement a priority-based scheduler.

The files kernel/param.h and kernel/proc.c were modified, and the functions effpriority() and scheduler() were changed in proc.c.

-kernel/param.h: #define ROUNDROBINSCH 1, #define PRIORITYSCH 2 and #define SCHEDULER PRIORITYSCH were added to choose the scheduler at compile time. To change the scheduler only the SCHEDULER line has to be changed and then make clean is run. The values start at 1 and not 0 because the preprocessor treats an unknown name as 0, so a typo would not match any scheduler. #define AGING was also added and was set to 0 for this task.

-kernel/proc.c, effpriority(): this function was added above scheduler(). It returns the effective priority of a process, which for this task is just its priority. It is a separate function so the aging policy can be changed later without changing the scheduler. It is static and inside #if SCHEDULER == PRIORITYSCH, so it is not compiled when round-robin is used, otherwise -Werror would stop the build because the function would not be used.

-kernel/proc.c, scheduler(): the original round-robin loop was kept inside #elif SCHEDULER == ROUNDROBINSCH, and the priority scheduler was added inside #if SCHEDULER == PRIORITYSCH. #else has an #error so the build stops if SCHEDULER has a wrong value. The priority scheduler first goes through all of proc[] and remembers the RUNNABLE process with the highest effective priority, taking and releasing each p->lock. If there is no RUNNABLE process it runs wfi to wait for an interrupt. Then it takes the lock of the chosen process again, checks that it is still RUNNABLE, sets it to RUNNING and switches to it with swtch(). The comparison uses > so when there is a tie the first process found in proc[] is chosen, which the assignment allows.

Three test programs were created in the user directory and added to UPROGS: schedtest1.c, schedtest2.c and schedtest3.c. In all three the parent sets its priority to 49 so it can create all the children before they start working, and then it calls wait(0) so the children compete for the CPU. The children use a work() function that only counts in a loop.

-schedulertest1: two children do the same amount of work, the first one with priority 10 and the second one with priority 30. The child with priority 30 finishes first even though it was created second.

-schedulertest2: a child with priority 10 and a child with priority 30 do the same work. When the child with priority 30 finishes, it turns itself into ps with exec(). ps shows the child with priority 10 as runnable with a big age, which means it never got the CPU while the higher priority child was running.
-schedulertest3: a child with priority 30 goes to sleep for 10 ticks with pause(10) while a child with priority 10 starts its work. When the high priority child wakes up, it gets the CPU in the middle of the work of the low priority child and prints the tick when it woke up before the low priority child finishes.
The tests were also run with SCHEDULER set to ROUNDROBINSCH to compare. With round-robin the two children of schedulertest1 finish close together and the age of the low priority child in schedulertest2 stays small, because the processes take turns.
All changes were committed and pushed.
Results:

```
xv6 kernel is booting

init: starting sh
$ schedulertest1
high priority (30) child finished
low priority (10) child finished
test 1 done
$ schedulertest2
high child finished its work, running ps:
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
6       sleeping        49      no aging        16384   2       schedulertest2
7       runnable        10      0       16384   6       schedulertest2
8       running         30      no aging        16384   6       ps
test 2 done
$ schedulertest3
hilow: started at tick 443
gh: going to sleep for 10 ticks
high: woke up at tick 454
low: finished at tick 467
test 3 done
$
```

```
xv6 kernel is booting

init: starting sh
$ schedulertest1
high priority (30) child finished
low priority (10) child finished
test 1 done
$ schedulertest2
high child finished its work, running ps:
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
6       sleeping        49      no aging        16384   2       schedulertest2
8       running         30      no aging        16384   6       ps
test 2 done
$ schedulertest3
high: going to sleep for 10 ticks
low: started at tick 654
high: woke up at tick 664
low: finished at tick 678
test 3 done
$
```

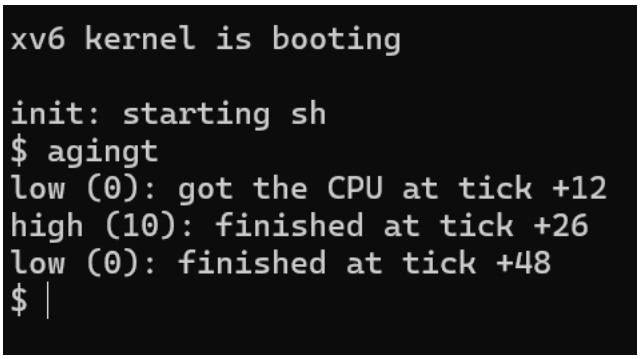
With this task I learned how the scheduler loop works and that the CPU always goes back to scheduler() after a process yields, sleeps or exits. I also learned how the preprocessor can keep two versions of the same code and compile only one of them with #if and #elif, and that the priority scheduler has to look at all the processes before choosing one, while round-robin runs the first RUNNABLE process it finds. Another thing I learned is that a priority scheduler can starve the processes with low priority, the shell with priority 0 did not respond while the tests were running because there was always a busy process with a higher priority.
The first difficulty was an implicit declaration of function effpriority error, because the function was below scheduler() and C needs a function to be defined before it is used. I fixed it by moving effpriority() above scheduler().

Task 5. (15 pts) Add aging to your priority-based scheduler.
The aging policy that was chosen is a threshold boost. A process keeps its normal priority until it has waited AGETHRESHOLD ticks in the ready queue. After that, 50 is added to its priority:

age = ticks - readytime
if age >= AGETHRESHOLD then effective priority = priority + AGEBOOST, else effective priority = priority

Since normal priorities are from 0 to 49, a process that was boosted (50 to 99) always beats every process that was not boosted, so no process can starve. Among the boosted processes, the one with the higher priority still wins, and the highest possible value is 49 + 50 = 99, which is MAXEFFPRIORITY. No age field was added, the effective priority is calculated from priority and readytime each time. When the boosted process runs and the timer makes it yield, readytime is set again, so its age goes back to 0 and it goes back to its normal priority. The threshold is 10 ticks, about one second in xv6, so a process does not wait much more than that

before it runs. A smaller threshold would act more like round-robin and a bigger one more like strict priority.
-kernel/param.h: the aging lines were replaced with #define AGING 1, #define AGETHRESHOLD 10 and #define AGEBOOST 50. AGING can still be set to 0 to compare with the priority scheduler without aging.
-kernel/proc.c, effpriority(): inside #if AGING the age is calculated as ticks - p->readytime and if it is at least AGETHRESHOLD, AGEBOOST is added to the priority, then the result is limited to MAXEFFPRIORITY. scheduler() did not have to be changed, since it already chooses the highest effective priority.
The programs pexec.c, agingt.c and schedulertest.c were created in the user directory and added to UPROGS.
-pexec: takes a priority, a command and its arguments. It calls setpriority() with the priority, then fork(), and the child runs the command with exec(argv[2], &argv[2]), so the command inherits the priority. The parent waits for it and prints the elapsed time (turnaround time) in ticks.
-agingtest: a child with priority 10 does a long work while a child with priority 0 waits. The low priority child prints the tick when it first gets the CPU. Without aging it only runs after the high priority child finishes, with aging it gets the CPU after waiting about AGETHRESHOLD ticks.
-schedeval: starts four children with the same amount of work and priorities 0, 5, 10 and 15. All of them wait on a pipe so they become RUNNABLE at the same moment. Each one measures its response time (first time it runs - start time) and its turnaround time (finish time - start time), sends them back to the parent with a pipe, and the parent prints the averages.
Results:



```
$ pexec 10 ps
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
6       sleeping        10      no aging        16384   2       pexec
7       running         10      no aging        16384   6       ps
$ pexec 10 ps
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
8       sleeping        10      no aging        16384   2       pexec
9       running         10      no aging        16384   8       ps
$ pexec 10 ps
pid     state   prio    age     size    ppid    name
1       sleeping        0       no aging        16384   0       init
2       sleeping        0       no aging        20480   1       sh
10      sleeping        10      no aging        16384   2       pexec
11      running         10      no aging        16384   10      ps
$
```

In the ps output, the age of the matmul with priority 0 goes up to about 10 and then goes back close to 0, because it gets boosted and runs. Without aging its age kept growing. The shell also kept responding while both matmul were running.

schedulertest was run with the round-robin scheduler, the priority scheduler without aging and the priority scheduler with aging:

Round robin test

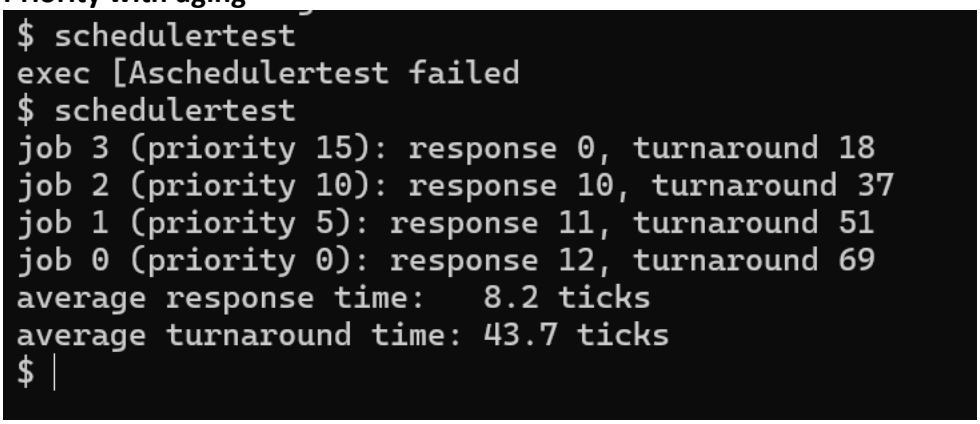
```
xv6 kernel is booting

init: starting sh
$ schedulertest
job 0 (priority 0): response 0, turnaround 60
job 2 (priority 10): response 2, turnaround 61
job 1 (priority 5): response 1, turnaround 62
job 3 (priority 15): response 3, turnaround 63
average response time:   1.5 ticks
average turnaround time: 61.5 ticks
$
```

Priority without aging

```
init: starting sh
$ schedulertest
job 3 (priority 15): response 0, turnaround 15
job 2 (priority 10): response 16, turnaround 37
job 1 (priority 5): response 37, turnaround 53
job 0 (priority 0): response 53, turnaround 69
average response time:   26.5 ticks
average turnaround time: 43.5 ticks
$
```

Priority with aging



Round-robin had the best response time because every job got a turn in one or two ticks, but the worst turnaround time because the four jobs shared the CPU the whole time and all of them finished near the end. The priority scheduler without aging had the best average turnaround time because the jobs ran one after the other, but the worst response time, the job with priority 0 had to wait until all the other jobs finished before running at all. With aging the response time of the low priority jobs was limited by the threshold, and the boosted jobs got their first turn in priority order. The turnaround time was between the other two, because the low priority jobs take some ticks from the high priority ones. Aging gives up a little turnaround time so that no process waits forever, and the higher priority jobs still finish first.
All changes were committed and pushed.
With this task I learned what starvation is and how aging fixes it, and that the policy only needs to change effpriority(), since the scheduler only compares the numbers. I also learned the difference between turnaround time and response time and that a scheduler can not be the best at both, round-robin is better for response time and strict priority is better for turnaround time, and aging is in the middle of both.
One difficulty was an "AGING" redefined error, because the old #define AGING 0 from task 4 was still in param.h when I added the new one, I fixed it by deleting the old line. Another difficulty was that the new effpriority() was pasted inside scheduler() over the priority loop, which gave several errors at the same time. I fixed it by restoring proc.c from my last commit with git checkout -- kernel/proc.c and pasting the function again in the right place. Choosing an aging policy was interesting, I compared linear and threshold aging before choosing the threshold policy because it is simple and it gives a clear limit on how long a process waits.

Task 6. (5 pts) Summary

One of the main things that I learnt with this programming assignment was how the scheduler decides which process runs next and how much that decision changes the behavior of the whole system. Adding a system call was faster this time because it follows the same path as wait2() in PA1, from user.h and usys.pl to syscall.c, sysproc.c, etc. I also learned how the kernel shares information with user programs, and that every field has to be added in the kernel, in pstat.h and in ps. Also, I was able to observe the pros and cons of scheduling. For example: the priority scheduler respects priorities but can starve processes, and aging fixes that at the cost of a little turnaround time.

I faced some difficulties while doing this assignment like a function below the place where it was called, or a #define written twice, and I learned to read the compiler messages and use grep to find where the problem was. Other errors came from differences between my version of xv6 and the older code, like %l in ps.c, and file names that can not be longer than 14 characters, which is why prioritytestprogram had to be renamed to prioritytest. Also, pushing the pa2 branch for the first time failed because it had no upstream branch, and I fixed it with git push -u origin pa2. The biggest challenge was understanding how locks and timer interrupts work together in the scheduler, and why some things like taking tickslock inside wakeup() would break the system.

AI use note: I used Claude to create a guide for explanations for this assignment, to help fixing Git problems, and for debugging and directing me in the right direction to make the programs work, particularly with the scheduler in task 4 and the aging policy in task 5. I wrote, built, and tested the code myself on my Linode VM.